



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO CIÊNCIAS DA COMPUTAÇÃO

Leonardo Fonseca Franchini - 23100484
Marcos Vinicius Moreira Machado - 23100478
Rita Louro Barbosa – 22203157

SO2

Entrega 5

1 Introdução

Este relatório descreve o progresso obtido na quinta etapa do projeto proposto pela disciplina de Sistemas Operacionais II.

Na Entrega 1, foram abordados a instalação e o preparo da infraestrutura necessária para viabilizar a comunicação entre sistemas independentes, cada um representado por uma máquina virtual QEMU, utilizando raw sockets para a transferência de frames Ethernet sem IP.

Na Entrega 2, foi abordada a implementação da comunicação entre componentes de um mesmo sistema autônomo (intra-VM), utilizando memória compartilhada no modelo Produtor × Consumidor com IPCs System V (shmget e semget), por meio da classe ShmEngine. Além disso, foi introduzida a multiplexação de portas para identificação dos componentes (sensores, atuadores, powertrain, gateway) dentro de uma mesma VM.

Na entrega 3, o objetivo foi garantir que todos os veículos compartilhem uma mesma noção de tempo, de forma que cada mensagem trocada na rede possa ser identificada de forma inequívoca pela combinação do seu endereço de origem com o instante em que foi gerada. Para isso, foi implementada uma versão simplificada do Precision Time Protocol (PTP – IEEE 1588), na qual uma unidade de infraestrutura externa chamada RSU (Road Side Unit) atua como referência de tempo para todos os veículos.

Na Entrega 4, o objetivo foi capacitar os sistemas autônomos a sincronizar suas posições relativas e se comunicar apenas com agentes do mesmo quadrante espacial. Para isso, foram implementados: um módulo de kernel que expõe as coordenadas de cada VM via /proc/position; a etiquetagem das mensagens com o quadrante de origem; e a expansão da infraestrutura de RSUs para quatro unidades, uma por quadrante, com troca automática de RSU conforme o veículo se desloca.

O foco desta Entrega 5 é organizar a troca de dados entre componentes em torno do tipo da informação, e não mais de portas fixas por componente. Para isso, foi introduzida a camada SmartData entre a aplicação e o Communicator, inspirada no Smart Data do EPOS e na codificação de unidades do padrão IEEE 1451. Cada dado passa a ser identificado de forma inequívoca por uma unidade (enum Unit), e as mensagens carregam o tipo de dado a que se referem. As portas deixam de identificar componentes e passam a identificar tipos de dados: o Protocol só notifica as camadas superiores quando o tipo da mensagem corresponde a um dado de interesse. Cada componente declara seus interesses (Interested_Smart_Data) e suas respostas (Responsive_Smart_Data), e a produção periódica de um dado só é iniciada quando existe um interesse correspondente.

2 Objetivos da P5

O principal objetivo desta entrega é reorganizar a comunicação em torno do tipo de dado trocado, introduzindo a camada SmartData entre a aplicação e o Communicator e

tornando a entrega de mensagens orientada a interesses. Para isso, os seguintes itens foram desenvolvidos:

- Criação da camada SmartData entre o Communicator e a aplicação, responsável por mediar o acesso aos dados em função do seu tipo.
- Definição do tipo Unit, que identifica os tipos de dados de forma inequívoca, com codificação de unidades inspirada no padrão IEEE 1451 e no Smart Data do EPOS.
- Adição dos campos de tipo de dado e tipo de mensagem em Message e no cabeçalho do Protocol, permitindo distinguir o dado transportado e diferenciar Interest, Response e mensagens PTP.
- Adoção de portas baseadas em tipos de dados: o Protocol passa a notificar apenas a porta correspondente ao tipo do dado da mensagem, descartando dados que não interessam às camadas superiores.
- Parametrização do Communicator por tipo de dado a enviar ou receber, com cada Communicator pertencendo a um InterestedSmartData ou a um ResponsiveSmartData.
- Implementação da lógica de declaração de interesse e de resposta periódica: ao receber um interesse de um período que ainda não atende, o ResponsiveSmartData cria uma nova thread que envia o dado periodicamente.
- Criação de novos componentes definidos a partir dos tipos de dados que consomem e produzem, em vez de papéis fixos.

3 Implementação

3.1 Informações gerais

O programa opera com múltiplas VMs simulando carros autônomos. O processo principal de cada VM — o gateway — inicializa as interfaces de rede e o protocolo de comunicação e, em seguida, cria processos filhos via fork() que simulam os componentes do veículo: CompA, CompB, CompC, CompD, CompE e TimeClient. A partir desta entrega, cada VM carrega o módulo de kernel position.ko, que expõe o quadrante atual via /proc/position. Todos os componentes da mesma VM leem esse arquivo, garantindo percepção compartilhada e consistente do espaço.

A partir desta entrega, a comunicação dos componentes passa a ser mediada pela camada SmartData, posicionada entre a aplicação e o Communicator. Cada SmartData encapsula um Communicator próprio, inscrito na porta correspondente ao tipo de dado que lhe interessa. O Communicator continua encapsulando o protocolo e o endereço lógico (par MAC + porta), utilizando o Protocol para empacotar e desempacotar dados e gerenciar o roteamento entre NICs, mas a porta deixa de identificar um componente e passa a identificar um tipo de dado. As mensagens carregam, além do timestamp e do quadrante de origem, o tipo do dado transportado e o tipo da mensagem (Interest, Response ou PTP).

3.2 Classes

Nesta seção detalharemos as classes do sistema, como elas funcionam e se relacionam, com ênfase nas novidades desta entrega.

3.2.1 RawSocketEngine

A classe RawSocketEngine é responsável pelo acesso de baixo nível ao hardware de rede, utilizando sockets RAW (AF_PACKET / SOCK_RAW) para enviar e receber frames Ethernet diretamente, sem passar pela pilha IP do sistema operacional. Durante sua inicialização, ela abre o socket com o EtherType customizado 0x8888, obtém o índice e o endereço MAC da interface via ioctl, realiza o bind na interface e configura um timeout de 100 ms para que o loop de recepção não bloqueie indefinidamente.

A engine lança uma thread dedicada que executa o loop de recepção em segundo plano. Quando um frame chega e passa pelo filtro de EtherType, essa thread chama `_handle`, implementado pela NIC<RawSocketEngine>, que copia o frame para um buffer do pool e notifica os observadores registrados. O envio é feito pelo método `_send`, que monta uma estrutura `sockaddr_ll` com o endereço de broadcast e transmite o frame via `sendto`.

3.2.2 NIC

A classe NIC é o núcleo da comunicação física do sistema, implementada como um template sobre uma engine (NIC<E>), combinando a interface abstrata NICBase com a engine concreta via herança privada. Essa separação permite reutilizar a mesma lógica de gerenciamento de buffers, endereçamento e notificação de observadores tanto para RawSocketEngine quanto para ShmEngine, apenas trocando o parâmetro de template.

A NIC mantém um pool interno de buffers para frames, evitando alocações dinâmicas frequentes. Ela implementa o padrão Observer, permitindo que protocolos se registrem para receber notificações de novos frames por EtherType. Um mutex protege o pool contra acesso concorrente entre a thread de recepção e o processo principal.

3.2.3 Protocol

A classe Protocol é responsável por empacotamento, desempacotamento e roteamento interno das mensagens. Ela define o formato dos pacotes adicionando um cabeçalho lógico (Protocol::Header) com portas de origem e destino, tamanho do payload e o campo `src_quadrant`, preenchido com o quadrante atual lido de `/proc/position` no momento do envio. O endereço MAC de origem é extraído diretamente do frame Ethernet, evitando redundância com o cabeçalho de enlace.

O Protocol observa a NIC e, ao receber um frame com o EtherType correto, executa o filtro de quadrante antes de qualquer roteamento. Apenas frames do mesmo quadrante prosseguem para a etapa de roteamento. A partir desta entrega, o roteamento é orientado ao tipo de dado: a porta de destino corresponde ao código da unidade (Unit) do dado, de modo que o Protocol notifica somente o Communicator inscrito naquele tipo. Mensagens cujo tipo não corresponde a nenhum interesse registrado não são entregues às camadas superiores. Para o envio, o Protocol seleciona a NIC adequada comparando

`expected_dst()` com o destino — mensagens inter-VM saem pelo raw socket e mensagens intra-VM saem pela memória compartilhada, de forma transparente para as camadas acima.

Para suporte ao unicast nas respostas PTP, o filtro de seleção de NIC foi refinado: a comparação com `expected_dst()` só é aplicada quando o destino não é vazio, permitindo que a `RawSocketEngine` aceite tanto destinos broadcast quanto unicast de forma transparente. O destino é broadcast — todas as RSUs recebem, e apenas a do quadrante correto aceita pelo filtro de `src_quadrant`.

3.2.4 Communicator

A classe `Communicator` é a interface de comunicação usada pela camada `SmartData`. Ao ser instanciado, ele se registra no Protocol para uma porta específica que, a partir desta entrega, corresponde ao código da unidade (`Unit`) do dado de interesse, de modo que apenas mensagens daquele tipo sejam recebidas. Cada `Communicator` pertence a um `InterestedSmartData` ou a um `ResponsiveSmartData`, ficando assim parametrizado pelo tipo de dado a enviar ou receber. Ele mantém uma lista interna de mensagens e um semáforo para sincronização.

O método `receive` bloqueia a thread chamadora por até 1 segundo aguardando a chegada de um novo dado. Quando `update` é chamado assincronamente pela thread de recepção, os dados do frame são copiados para um objeto `Message` alocado em heap, inserido na lista interna, e o semáforo é liberado, acordando a thread bloqueada. Esse mecanismo promove o processamento desacoplado e assíncrono das mensagens.

3.2.5 Message

A classe `Message` representa a unidade de dados trocada entre os agentes. Ela possui os campos `_data` (buffer de até 1024 bytes), `_size` (tamanho válido dos dados), `_src` (endereço MAC de origem, preenchido pelo `Communicator`), `_origin` (quadrante de origem, 2 bits, preenchido via `set_origin()` com o valor de `Position::quadrant()` no momento do envio) e `timestamp` (`uint64_t` em nanossegundos, preenchido com `clock_gettime(CLOCK_REALTIME, ...)`). A partir desta entrega, as mensagens de `Interest` e `Response` são serializadas diretamente no payload (`_data`) via as structs `InterestMsg` e `ResponseMsg`, que contêm os campos `origin`, `timestamp`, `type`, `period` (para `Interest`) ou `value` (para `Response`), e um campo `kind` para distinguir `Interest` de `Response`. A camada `SmartData` inspeciona o payload para identificar o tipo e a natureza de cada mensagem recebida.

3.2.6 PTPFrame

A struct `PTPFrame` representa o payload das mensagens de sincronização temporal. Ela é composta por `message_type` (`uint8_t`), que indica a etapa do fluxo PTP, e `timestamp` (`uint64_t`), com o instante de tempo relevante para aquela etapa. Os valores de `message_type` são:

Valor	Tipo	Descrição
0	Sync Request	Enviado pelo slave para iniciar a sincronização
1	Sync Response	Enviado pelo master com o timestamp T1
2	Delay_Req	Enviado pelo slave para medir o atraso de rede
3	Delay_Resp	Enviado pelo master com o timestamp T4

A struct também contém o campo `seq (uint32_t)`, um número de sequência único por veículo gerado a partir do último byte do MAC (`mac.bytes[5] << 24 | counter`), que permite ao veículo correlacionar cada resposta da RSU com o request correspondente, descartando respostas destinadas a outros veículos.

A struct é marcada com `__attribute__((packed))` para garantir ausência de padding, permitindo que seja copiada diretamente para o payload de um Message e transmitida pela rede sem inconsistências.

3.2.7 RSU (Road Side Unit)

Nesta entrega, o sistema passa a operar com quatro RSUs, uma por quadrante. Cada instância é executada em uma VM dedicada com um MAC fixo. Cada RSU recebe seu quadrante como parâmetro no construtor, passado via linha de comando (`quadrant=N` no `cmdline` do `kernel`). Ao inicializar, chama `Protocol::set_quadrant(_quadrant)`. A classe RSU cumpre o papel de master PTP e possui uma `NIC<RawSocketEngine>`, um `Protocol` e um `Communicator` registrado na porta reservada 9999. Ao receber uma mensagem, a RSU verifica se o quadrante do remetente corresponde ao seu; caso contrário, descarta a mensagem. Para mensagens válidas, age conforme o `message_type`:

- **Sync Request (0):** registra o instante atual como T1 e responde com `{type=1, timestamp=T1}`.
- **Delay_Req (2):** registra o instante de recebimento como T4 e responde com `{type=3, timestamp=T4}`.

As respostas são enviadas em unicast diretamente para o MAC do veículo que originou o request, extraído via `msg.src()` do frame Ethernet recebido. Isso evita flooding na rede e segue o modelo unicast on-demand definido pelo IEEE 1588-2008 Annex D, recomendado para redes veiculares onde broadcast não escala.

3.2.8 TimeClient

O TimeClient é o componente de cada veículo responsável por sincronizar o relógio da VM com a RSU e disponibilizá-lo aos demais componentes. Ele herda de Component (possuindo uma NIC<ShmEngine> para comunicação intra-VM) e possui adicionalmente uma NIC<RawSocketEngine> própria, anexada ao mesmo Protocol, para se comunicar diretamente com a RSU pela rede.

O método syncTime implementa o fluxo PTP do lado slave:

1. Envia PTPFrame{type=0} para a RSU e aguarda resposta.
2. Ao receber {type=1, T1}, registra o instante de recebimento como T2.
3. Envia PTPFrame{type=2} (Delay_Req) e registra o instante de envio como T3.
4. Ao receber {type=3, T4}, calcula:
 - $\text{delay} = ((T2 - T1) + (T4 - T3)) / 2$
 - $\text{offset} = ((T2 - T1) - (T4 - T3)) / 2$
5. Ajusta o relógio local via clock_settime(CLOCK_REALTIME, ...) somando o offset ao tempo atual.

Em vez de aplicar o offset inteiro de uma vez — o que causaria saltos abruptos no relógio e corromperia medições futuras — aplica-se $\text{correction} = -\text{offset}$ a cada ciclo, ajustando o relógio diretamente pelo offset calculado.

O método run repete esse ciclo a cada 500 ms. Como todos os processos de uma mesma VM compartilham o relógio do sistema operacional, o ajuste feito pelo TimeClient via clock_settime é imediatamente visível para todos os outros componentes daquele veículo, sem necessidade de comunicação adicional.

3.2.9 SmartData

A classe SmartData é a base da nova camada introduzida nesta entrega, posicionada entre a aplicação e o Communicator e inspirada no Smart Data do EPOS e na codificação de unidades do padrão IEEE 1451. Ela define a interface comum dos dados inteligentes — os métodos start() e stop(), que controlam o ciclo de vida do processamento — e abriga a definição de unidades por meio da classe aninhada Unit. As especializações concretas dessa camada são o InterestedSmartData, para dados que o componente deseja consumir, e o ResponsiveSmartData, para dados que o componente produz.

A classe Unit codifica o tipo de um dado em um inteiro de 32 bits, identificando-o de forma inequívoca. O layout reserva um bit para indicar o uso do Sistema Internacional, dois bits para o tipo numérico (I32, I64, F32 ou D64), dois bits para o modo de combinação das unidades (direto, divisão, logaritmo) e três bits para o expoente de cada uma das grandezas SI base (segundo, radiano, metro, quilograma, ampere, kelvin, mol e candela). A partir dessa codificação, são definidas grandezas físicas prontas para uso no projeto, como Temperature, Acceleration, Velocity, Pressure e Force, entre outras. Um auxiliar de metaprogramação (Get) recupera, em tempo de compilação, o tipo numérico C++ correspondente a cada unidade.

3.2.10 InterestedSmartData

O InterestedSmartData é a especialização da camada SmartData responsável por declarar interesse em um tipo de dado. É um template parametrizado pelo código da

unidade (Unit) desejada e recebe, em sua construção, o período com que deseja receber o dado. Internamente, ele encapsula um Communicator inscrito na porta correspondente ao tipo de dado de interesse.

Ao ser iniciado, o InterestedSmartData lança uma thread própria que envia uma mensagem de Interest em broadcast — com o tipo de mensagem 0, o código da unidade e o período desejado no payload — e passa a aguardar as respostas. A cada Response recebido, registra o instante de chegada. Caso fique um intervalo prolongado sem receber respostas, reenvia o Interest, garantindo que o fluxo de dados seja restabelecido mesmo que um produtor entre na rede ou se recupere posteriormente.

3.2.11 ResponsiveSmartData

O ResponsiveSmartData é a especialização da camada SmartData responsável por produzir e disponibilizar um tipo de dado. É um template parametrizado por um Transducer, do qual herda o código da unidade produzida, e encapsula um Communicator inscrito na porta correspondente a esse tipo de dado.

Ao ser iniciado, o ResponsiveSmartData lança uma thread que aguarda mensagens de Interest. Quando recebe um interesse cujo período ainda não está sendo atendido, registra esse período e cria uma nova PeriodicThread dedicada que, a cada ciclo, lê o valor do Transducer e o envia como Response. Dessa forma, cada período distinto de interesse é atendido por uma thread independente, e um mesmo período não gera envios duplicados.

3.2.12 Transducer

O Transducer modela a fonte física de um dado, isto é, o sensor que mede uma grandeza do ambiente. É um template parametrizado pelo código da unidade que mede, expondo o método estático sense(), que retorna o valor amostrado no tipo numérico correspondente àquela unidade. As implementações concretas desta entrega são: Thermometer (temperatura), Accelerometer (aceleração), Velocimeter (velocidade), PressureSensor (pressão), VoltageSensor (voltagem) e CurrentSensor (corrente); todos retornam valores fixos para fins de teste.

3.2.13 PeriodicThread

A classe PeriodicThread executa uma tarefa em intervalos regulares numa thread própria, sendo o mecanismo usado pelo ResponsiveSmartData para enviar respostas periodicamente. O período é informado na construção, em microssegundos, e a thread inicia automaticamente. A cada ciclo, a tarefa é executada e a thread aguarda até o próximo instante previsto; se a tarefa demorar mais do que o período, o próximo disparo ocorre imediatamente, sem acumular execuções atrasadas. O período pode ser alterado em tempo de execução, e a parada é segura e idempotente, ocorrendo na destruição do objeto.

3.2.14 Componentes baseados em tipos de dados

Com a camada SmartData, os componentes deixam de ser definidos por papéis fixos e passam a ser descritos pelos tipos de dados que consomem e produzem. Cada componente herda de Component e, no método add_smart_dadas(), registra um InterestedSmartData para cada dado que deseja receber e um ResponsiveSmartData para cada dado que produz. Como exemplo desta entrega, CompA e CompB trocam temperatura e aceleração entre si; CompC e CompD trocam velocidade e pressão;

CompE envia voltagem e declara interesse em corrente, demonstrando o comportamento do sistema na ausência de um produtor correspondente.

3.3 Fluxo de dados

Envio

O componente preenche um objeto Message com dados, tamanho, endereço de origem e timestamp, e o repassa ao Communicator. Este chama Protocol::send, que seleciona a NIC adequada com base no destino, aloca um buffer do pool, monta o cabeçalho de protocolo e copia o payload. A engine correspondente — RawSocketEngine para inter-VM ou ShmEngine para intra-VM — realiza a transmissão efetiva. Após o envio, o buffer é devolvido imediatamente ao pool.

Recebimento

Na RawSocketEngine, uma thread dedicada aguarda frames no socket com timeout, filtra pelo EtherType 0x8888 e os entrega à NIC via _handle. Na ShmEngine, a chegada de um novo frame é sinalizada por SIGUSR1, disparado pelo processo que escreveu na memória compartilhada (ver seção 3.4).

Em ambos os casos, a NIC copia o frame para um buffer livre do pool e notifica o Protocol via Observed::notify. O Protocol extrai a porta de destino e notifica o Communicator registrado. O Communicator copia os dados para um Message em heap, insere na lista interna e libera o semáforo, desbloqueando a thread em receive. Após a extração, o buffer é devolvido ao pool.

3.4 Signal Handler

O tratamento de sinais é o mecanismo de notificação assíncrona da comunicação intra-VM. Quando um processo escreve um frame no ShmChannel, ele itera sobre os PIDs dos leitores registrados e envia kill(pid, SIGUSR1) para cada um, exceto para si mesmo.

O handler estático _signal_handler acessa a instância da ShmEngine via ponteiro global e chama _try_read, que lê todos os frames pendentes comparando o head individual com o tail global. Cada frame lido é entregue via _handle. O handler é registrado com sigaction, que permite definir com precisão a função chamada, os sinais bloqueados durante a execução e as flags de comportamento, garantindo tratamento seguro e previsível.

3.5 ShmEngine

A ShmEngine implementa a comunicação intra-VM usando memória compartilhada System V no modelo Produtor-Consumidor. No construtor, cria ou abre um segmento de memória via shmget/shmat e um semáforo System V via semget para proteger a seção crítica de escrita. O processo se registra como leitor no ShmChannel via enroll(getpid()), que atribui um slot no array de PIDs e inicializa o ponteiro de leitura individual.

O ShmChannel implementa uma fila circular de 8 slots de frames Ethernet. Cada leitor possui seu próprio head, de modo que múltiplos processos possam ler de forma

independente sem interferência — equivalente a multicast dentro da VM. O tail, compartilhado entre todos, avança a cada novo frame inserido.

3.6 Endereçamento

O endereço completo de cada agente é o par {MAC, porta}, estruturado como Protocol::Address. A partir desta entrega, a semântica da porta muda: em vez de identificar um tipo de componente, ela passa a identificar um tipo de dado, recebendo o código da unidade (Unit) correspondente. Assim, cada SmartData se inscreve na porta do dado que lhe interessa, e o Protocol entrega a mensagem apenas a quem se registrou naquele tipo. As portas fixas anteriores (gateway, sensor, atuador, powertrain, TimeClient) permanecem reservadas para os fluxos de controle e de sincronização que ainda dependem de papéis, como a porta da RSU usada pelo PTP. Como o MAC é único por VM, o par {MAC, porta} continua identificando cada fluxo de forma globalmente única, sem depender de PIDs — que não cruzam fronteiras de VM e mudam a cada execução.

Componente	Porta
Gateway	1000
Powertrain	1003
TimeClient	1004
RSU	9999

3.7 Precision Time Protocol (PTP)

O Precision Time Protocol (PTP), definido pelo padrão IEEE 1588, é um protocolo de sincronização de relógios projetado para redes de computadores. Seus principais conceitos são:

- **Dois tipos de entidade:** mestre (*master*) e escravo (*slave*). O mestre é a referência de tempo da rede; os escravos ajustam seus relógios com base nas mensagens trocadas com ele.
- O protocolo funciona por troca de mensagens com timestamps: o mestre envia o instante em que transmitiu a mensagem (T1), o escravo registra o instante em que a recebeu (T2), depois envia uma requisição de delay e registra o instante de envio (T3), e o mestre registra o instante em que a recebeu (T4).
- Com os quatro timestamps, o escravo consegue calcular o atraso de propagação da rede e o offset entre seu relógio e o do mestre, ajustando seu relógio local de forma precisa.
- A escolha de qual agente atua como mestre pode ser arbitrária. Neste projeto, a RSU cumpre esse papel.

Nesta implementação, o protocolo foi simplificado para o contexto do projeto, mantendo o fluxo de quatro timestamps (Sync, Sync Response, Delay_Req e Delay_Resp) e o cálculo de offset e delay, mas sem os mecanismos de eleição de mestre e anúncio periódico presentes no padrão completo.

3.8 Comunicação entre VMs, sincronização temporal e troca de RSU

A comunicação inter-VM é realizada pela RawSocketEngine usando frames Ethernet raw com EtherType 0x8888, sem camada IP. A partir desta entrega, os veículos enviam sempre em broadcast. O TimeClient envia mensagens PTP com src_quadrant atualizado; a RSU do quadrante correspondente responde e as demais descartam pelo filtro da NIC. O Protocol garante o roteamento correto entre RawSocketEngine e ShmEngine de forma transparente para as camadas acima, conforme descrito na seção 3.2.3.

A sincronização temporal e a localização espacial integram esses mecanismos: o TimeClient usa a RawSocketEngine para trocar mensagens PTP com a RSU do quadrante atual, e o resultado do ajuste via clock_settime é automaticamente compartilhado com todos os componentes da mesma VM pelo relógio do sistema operacional. A troca de RSU é implícita. O veículo envia sempre em broadcast com src_quadrant atualizado via Position::quadrant(). A NIC de cada RSU filtra pelo quadrante via Protocol::verifica_quadrante(), aceitando apenas frames do seu quadrante. Não existe handoff explícito.

3.9 Módulo de kernel: position.ko

O módulo de kernel position.ko implementa a interface de sincronização espacial exigida pela entrega. Ao ser carregado (insmod), ele cria a entrada /proc/position com permissão somente-leitura (0444) e configura um timer do kernel com período de 4 segundos. A cada disparo do timer, o campo quadrante (uint8_t) é atualizado com um valor aleatório entre 0 e 3, obtido via get_random_u32_below(4). A leitura é servida pelo handler proc_read, que copia o valor atual para o buffer do usuário via copy_to_user. O módulo é descarregado via rmmod, que cancela o timer com timer_delete_sync e remove a entrada /proc.

O acesso ao módulo pelo userspace é encapsulado pela classe Position (include/utils/position.hpp), que abre /proc/position, lê o inteiro e retorna q & 0x3. Como todos os processos de uma mesma VM leem o mesmo arquivo no kernel, a percepção do quadrante é idêntica para todos os componentes daquele sistema autônomo, cumprindo o requisito de percepção espacial compartilhada.

3.10 Cross-layer design:

A NIC e o Protocol implementam um design cross-layer: a NIC acessa o método privado estático Protocol::verifica_quadrante() via declaração friend, inspecionando o src_quadrant do Protocol::Header sem expor esse método publicamente. Isso permite que a filtragem ocorra na camada mais baixa possível (NIC,

na subida), enquanto o Protocol mantém controle sobre a lógica de quadrante via `set_quadrant` e `accept`.

3.11 Arquitetura da camada SmartData

A figura a seguir resume o posicionamento da camada SmartData na pilha de comunicação. Na subida, a NIC entrega o frame ao Protocol apenas se o quadrante de origem corresponder ao quadrante local. O Protocol, por sua vez, notifica somente a porta associada ao tipo de dado da mensagem, de modo que cada Communicator — e o SmartData que o encapsula — recebe exclusivamente os dados do tipo em que se inscreveu. Assim, um mesmo Protocol pode atender vários SmartData simultaneamente, cada um responsável por um tipo de dado distinto.

